

第四章 语法分析——自顶向下方法

考点速查手册（第一讲 + 第二三讲）

涵盖：存在的问题与CFG改造 · FIRST/FOLLOW集 · LL(1)文法 · 预测分析器 · 递归子程序法

一、语法分析器概览 (Syntax Analyzer Overview)

核心功能：检查Token序列是否符合文法，判断输入串是否为文法的一个句子。

自顶向下基本思想：从文法开始符号出发，寻求输入串的**最左推导**；从根节点开始构造语法树。

分析方法：自顶向下（递归子程序法、预测分析法LL(1)）；自底向上（算符优先、LR分析法）。

二、自顶向下分析面临的问题 [P7-11 第一讲]

2.1 问题1：回溯 (Backtracking)

原因：多个候选式左端第一个符号相同，分析程序无法根据当前输入符号唯一选择产生式，只能试探。

例： $A \rightarrow ab \mid a$ ，面对输入 "a"，无法区分两个候选式，需要回溯。[P6]

2.2 问题2：左递归 (Left Recursion)

直接左递归： $A \rightarrow A\alpha$ （文法含 $A \Rightarrow A\alpha$ ）；**间接左递归：** $A \Rightarrow^+ A\alpha$

后果：陷入无限循环，无法终止。[P9]

例： $S \rightarrow Say \mid *$ ，匹配 " $*ayay$ " 时， $S \Rightarrow Say \Rightarrow Sayay \Rightarrow \dots$ ，永不停止。

三、CFG 改造方法 [P13-23 第一讲]

3.1 消除二义性 (Eliminating Ambiguity)

算术表达式例 [P13]:

二义性: $E \rightarrow E+E \mid E-E \mid E^*E \mid E/E \mid (E) \mid id$

无二义: $E \rightarrow E+T \mid E-T \mid T$; $T \rightarrow T^*F \mid T/F \mid F$; $F \rightarrow (E) \mid id$

if-else 悬挂问题 (dangling else) [P14-16]: 每个 else 与**最近的未配对的 then** 配对:

$$S \rightarrow U \mid M$$

$$U \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } M \text{ else } U$$

$$M \rightarrow \text{if } E \text{ then } M \text{ else } M \mid \text{other}$$

3.2 消除左递归 (Eliminating Left Recursion) [P18-21]

直接左递归消除公式：

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_n \mid \beta_1 \mid \dots \mid \beta_m \quad \text{替换为:}$$

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_m A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_n A' \mid \varepsilon$$

例： $E \rightarrow E+T \mid T \Rightarrow E \rightarrow TE', E' \rightarrow +TE' \mid \varepsilon$

间接左递归 [P20]：为非终结符编号，将低编号定义代入高编号产生式，化为直接左递归后消除，删多余产生式。

3.3 提取左因子 (Left-Factor Extraction) [P22-23]

公式： $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \mid \alpha\beta_n \mid \gamma_1 \mid \dots \mid \gamma_m$ 替换为：

$$A \rightarrow \alpha A' \mid \gamma_1 \mid \dots \mid \gamma_m \quad A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

例：if语句 $\Rightarrow S \rightarrow \text{if } E \text{ then } SS' \mid \text{other}; S' \rightarrow \varepsilon \mid \text{else } S$

四、FIRST 集与 FOLLOW 集 [P9-17 第二三讲]

4.1 FIRST 集 (First Set)

定义： $\text{FIRST}(\alpha) = \{a \mid \alpha \Rightarrow^* a \dots, a \in V_T\}$ 。若 $\alpha \Rightarrow^* \varepsilon$ ，则 $\varepsilon \in \text{FIRST}(\alpha)$ 。

求 $\text{FIRST}(X)$ 的规则 [P10-11]：

- $X \in V_T$ ： $\text{FIRST}(X) = \{X\}$
- $X \in V_N$ ， $X \rightarrow a \dots \in P \Rightarrow a \in \text{FIRST}(X)$ ； $X \rightarrow \varepsilon \in P \Rightarrow \varepsilon \in \text{FIRST}(X)$
- $X \rightarrow Y_1 Y_2 \dots Y_k \in P$ ：依次加入 $\text{FIRST}(Y_i) - \{\varepsilon\}$ ；若 $\varepsilon \in \text{FIRST}(Y_i)$ 则继续 Y_{i+1} ；若全部可推 ε 则 $\varepsilon \in \text{FIRST}(X)$
- 重复直到不变

求 $\text{FIRST}(\alpha)$ ， $\alpha = X_1 X_2 \dots X_n$ [P13]：

- 初值： $\text{FIRST}(\alpha) = \text{FIRST}(X_1) - \{\varepsilon\}$ ， $k=1$
- while $\varepsilon \in \text{FIRST}(X_k)$ and $k < n$ ： $\text{FIRST}(\alpha) += \text{FIRST}(X_{k+1}) - \{\varepsilon\}$ ； $k++$
- if $k=n$ and $\varepsilon \in \text{FIRST}(X_k)$ ： $\varepsilon \in \text{FIRST}(\alpha)$

表达式文法 FIRST 集 [P12]：

F	T	E	E'	T'
{(, id}	{(, id}	{(, id}	{+, ε}	{*, ε}

4.2 FOLLOW 集 (Follow Set)

定义： $\text{FOLLOW}(A) = \{a \mid S \Rightarrow^* \dots Aa \dots, a \in V_T\}$ 。若 $S \Rightarrow^* \dots A$ ，则 $\$ \in \text{FOLLOW}(A)$ 。

求 FOLLOW 集的规则 [P15-16]：

- 若 A 为开始符，则 $\$ \in \text{FOLLOW}(A)$
- 若 $B \rightarrow \alpha A \beta \in P$ ，则 $\text{FIRST}(\beta) - \{\epsilon\} \subseteq \text{FOLLOW}(A)$
- 若 $B \rightarrow \alpha A$ 或 $B \rightarrow \alpha A \beta$ 且 $\epsilon \in \text{FIRST}(\beta)$ ，则 $\text{FOLLOW}(B) \subseteq \text{FOLLOW}(A)$
- 重复直到不变

表达式文法 FOLLOW 集 [P17]：

E	E'	T	T'	F
{\$, }	{\$, }	{+, }, \$}	{+, }, \$}	{*, +, }, \$}

五、LL(1) 文法 [P18-20 第二三讲]

定义：对 G 中任意变量 A，设 $A \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_n$ 是 A 的所有产生式，若满足：

- (1) $\text{FIRST}(\alpha_i) \cap \text{FIRST}(\alpha_j) = \emptyset \quad (i \neq j)$
- (2) 若 $\epsilon \in \text{FIRST}(\alpha_i)$ ，则 $\text{FOLLOW}(A) \cap \text{FIRST}(\alpha_i) = \emptyset$

则称 G 为 **LL(1) 文法**。

含义：第1个L=从**左**向右扫描；第2个L=生成**最左**推导；1=读入**1**个符号可唯一确定下一步推导。

验证例（表达式文法） [P19]：

- E'： $\text{FIRST}(+TE') = \{+\}$ ， ϵ 的 $\text{FIRST} = \{\epsilon\}$ ，且 $+ \notin \text{FOLLOW}(E') = \{, \}$ ， $\$ \notin \text{FOLLOW}(E')$ ✓
- T'： $\text{FIRST}(*FT') = \{*\}$ ， $*$ $\notin \text{FOLLOW}(T') = \{+, \}$ ， $\$ \notin \text{FOLLOW}(T')$ ✓
- F： $\text{FIRST}((E)) = \{(, \}$ ， $\text{FIRST}(\text{id}) = \{\text{id}\}$ ，两者不相交 ✓

非LL(1)示例 [P20]： $A \rightarrow ab|a$ ， $\text{FIRST}(ab) = \{a\} = \text{FIRST}(a)$ ，条件(1)不满足 → 非LL(1)。

六、预测分析器（LL(1)分析器） [P22-46 第二三讲]

6.1 系统组成 [P22-23]

- **输入缓冲区：**Token序列，\$为结束符
- **分析栈：**栈底为\$，初始压入开始符；栈内容 = 当前句型中**已分析完终结字符串之后的剩余部分** [P27]
- **预测分析表 M[A,a]：**行=非终结符，列=终结符；存放产生式或错误标志
- **总控程序：**控制分析流程，输出产生式序列

6.2 LL(1) 通用控制算法 [P34-35]

```
初始: $ 和开始符号入栈; 读入第一个字符 a
repeat
  X = 当前栈顶符号; a = 当前输入符号
  if  $X \in V_T \cup \{\$\}$  then
    if  $X = a$  then
      if  $X \neq \$$  then {弹出X, 前移输入指针}
      else {分析成功, 结束}
    else error
  else // X 为非终结符
    if  $M[X, a] = Y_1 Y_2 \cdots Y_k$  then
      {弹出X; 依次将 $Y_k \cdots Y_2 Y_1$ 压栈; 输出  $X \rightarrow Y_1 \cdots Y_k$ }
    else error
until  $X = \$$ 
```

6.3 预测分析表构造算法 [P37]

对每个产生式 $A \rightarrow \alpha$:

- 对每个 $a \in \text{FIRST}(\alpha)$ ($a \neq \epsilon$) , 将 $A \rightarrow \alpha$ 填入 $M[A, a]$
- 若 $\epsilon \in \text{FIRST}(\alpha)$, 则对每个 $b \in \text{FOLLOW}(A)$ (含 $\$$) , 将 $A \rightarrow \alpha$ 填入 $M[A, b]$
- 所有未定义的 $M[A, a]$ 标上**错误标志**
- **若某格有两个产生式填入 $\rightarrow$ 该文法非LL(1)!**

表达式文法预测分析表 (含synch错误恢复) [P37,44]:

非终结符	id	+	*	(	)	\$
E	$\rightarrow TE'$	—	—	$\rightarrow TE'$	synch	synch
E'	—	$\rightarrow +TE'$	—	—	$\rightarrow \epsilon$	$\rightarrow \epsilon$
T	$\rightarrow FT'$	synch	—	$\rightarrow FT'$	synch	synch
T'	—	$\rightarrow \epsilon$	$\rightarrow *FT'$	—	$\rightarrow \epsilon$	$\rightarrow \epsilon$
F	$\rightarrow \text{id}$	synch	synch	$\rightarrow (E)$	synch	synch

6.4 预测分析法完整步骤 [P39]

1. 构造文法
2. **改造文法**: 消除二义性、消除左递归、提取左因子
3. 求每个候选式的 FIRST 集和每个变量的 FOLLOW 集
4. 检查是否为 LL(1) 文法
5. 构造预测分析表 M
6. 实现预测分析器

6.5 出错处理 (Error Recovery) [P40-46]

恐慌模式 (Panic Mode) [P42]: 忽略输入符号直到出现**同步词法单元 (synch)**, 弹出栈顶非终结符, 继续分析。

短语层次恢复 (Phrase-Level) [P46]: 在分析表空白处插入错误处理例程, 可修改/插入/删除输入符号。

同步集合选取原则 [P43]:

- FOLLOW(A)中终结符 $\subseteq$ A的同步集合
- 高层结构开始符 (if、while等) 加入低层结构同步集合
- FIRST(A)中终结符也加入同步集合
- 若A能推出 ϵ , 出错时可用 $A \rightarrow \epsilon$ 作默认产生式
- 终结符在栈顶不匹配时, 直接**弹出**该终结符

七、语法图与递归子程序法 [P47-59 第二三讲]

7.1 语法图 (Syntax Diagram) [P48-52]

每个非终结符对应一个图; 终结符用椭圆, 非终结符用方框; 非终结符上的转换 = 调用对应子程序。

化简 [P51]: $E \rightarrow TE'$, $E' \rightarrow +TE' | \epsilon$ 合并为循环结构 $E \rightarrow T(+T)^*$; 同理 $T \rightarrow F(*F)^*$ 。

7.2 递归子程序法 (Recursive Descent Parsing) [P53-58]

表达式文法递归子程序 (Pascal风格) :

```
procedure E;    {  $E \rightarrow T(+T)^*$  }
begin
  T;
  while lookahead='+' do begin match('+'); T end
end;

procedure T;    {  $T \rightarrow F(*F)^*$  }
begin
  F;
  while lookahead='*' do begin match('*'); F end
end;

procedure F;    {  $F \rightarrow (E) | id$  }
begin
  if lookahead='(' then begin match('('); E; match(')') end
  else if lookahead=id then match(id)
  else error
end;

procedure match(t);  { 服务子程序 }
begin
  if lookahead=t then lookahead:=nexttoken else error
end;
```

优点	缺点
直观、简单、可读性好; 便于扩充	递归实现效率低; 处理能力有限; 通用性差, 难以自动生成

八、关键考点与公式速查

1. 消除直接左递归: $A \rightarrow A\alpha | \beta \implies A \rightarrow \beta A', A' \rightarrow \alpha A' | \varepsilon$
2. 提取左因子: $A \rightarrow \alpha\beta_1 | \alpha\beta_2 \implies A \rightarrow \alpha A', A' \rightarrow \beta_1 | \beta_2$
3. 填预测分析表: 产生式 $A \rightarrow \alpha$: 遍历 $\text{FIRST}(\alpha)$ 中终结符填 $M[A, \cdot]$; 若 $\varepsilon \in \text{FIRST}(\alpha)$ 再遍历 $\text{FOLLOW}(A)$ 填 $M[A, \cdot]$
4. 验证LL(1): 对每个A的所有候选式检验两个条件; 若某格有冲突 (两个产生式) $\rightarrow$ **非LL(1)**
5. 分析动作:
 - 栈顶非终结符X, 输入a $\rightarrow$ 查 $M[X, a]$ 展开
 - 栈顶终结符X=a $\rightarrow$ 弹出, 移进
 - 栈顶X $\neq$ a $\rightarrow$ 出错
 - 栈顶\$, 输入\$ $\rightarrow$ **分析成功**

文法类型与适用范围 [P24-25 第一讲]: $\text{LL}(1)\text{文法} \subsetneq \text{LR文法} \subsetneq \text{CFG}$

- LL文法 $\rightarrow$ 递归下降/预测分析 (手工实现首选)
- LR文法 $\rightarrow$ LR分析法 (自动生成工具首选)
- **没有一种方法能有效分析所有CFG**
- LL(1)文法必须: 无二义性、无左递归、无公共左因子